

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NHẬP MÔN HỌC MÁY

ĐỒ ÁN CUỐI KÌ

Người hướng dẫn: **PGS TS LÊ ANH CƯỜNG**

Người thực hiện: **LÊ TUẤN THÀNH - 52100312**

Khoá: 25

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2023

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NHẬP MÔN HỌC MÁY

ĐỒ ÁN CUỐI KÌ

Người hướng dẫn: **PGS TS LÊ ANH CƯỜNG**

Người thực hiện: **LÊ TUẤN THÀNH - 52100312**

Khoá: 25

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2023

LỜI CẢM ƠN

Để hoàn thành bài báo cáo này, em xin cảm ơn đến PGS TS Lê Anh Cường đã tận tình hướng dẫn cho em đi từ những cái cơ bản nhất của môn Nhập môn Học máy để em có thể có đủ kiến thức hoàn thành bài báo cáo cuối kỳ này, em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 10 tháng 12 năm 2023

Tác giả

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA GIẢNG VIÊN

Tp Hồ Chí Minh, ngày... tháng... năm...
(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN	1
NHẬN XÉT CỦA GIẢNG VIÊN	2
MỤC LỤC.....	3
DANH MỤC HÌNH ẢNH.....	4
DANH MỤC TỪ VIẾT TẮT.....	5
1. Định nghĩa và ác phương pháp Optimizer trong huấn luyện mô hình học máy.....	6
1.1 Định nghĩa Optimizer	6
1.2 Các phương pháp Optimizers.....	6
1.2.1 Gradient Descent (GD)	6
1.2.2 Stochastic Gradient Descent (SGD)	9
1.2.3 Momentum	10
1.2.4 Adagrad	13
1.2.5 RMSprop	14
1.2.6 Adam	14
2. Continual Learning và Test Production khi xây dựng một giải pháp học máy.....	15
2.1 Continual Learning	15
2.1.1 Định nghĩa.....	15
2.1.2 Thách Thức và Giải Pháp.....	16
2.1.3 Các phương pháp Continual Learning	16
2.2 Test Production.....	17
2.2.1 Định nghĩa.....	17
2.2.2 Bước Thiết Kế Test Production	17
2.2.3 Lợi ích của Test Production.....	18
2.3 Tổng kết	18
TÀI LIỆU THAM KHẢO	19

DANH MỤC HÌNH ẢNH

Hình 1. <i>Gradient Descent</i>	8
Hình 2. <i>Stochastic Gradient Descent và Gradient Descent</i>	9
Hình 3. <i>GD và GD with Momentum</i>	10
Hình 4 <i>GD with Momemtum</i>	12
Hình 5. <i>Công thức Adagrad</i>	13
Hình 6. <i>Công thức RMSprop</i>	14
Hình 7. <i>Các thuật toán optimizers</i>	15

DANH MỤC TỪ VIẾT TẮT

GD: Gradient Descent

SGD: Stochastic Gradient Descent

CL: Continual Learning

1. Định nghĩa và các phương pháp Optimizer trong huấn luyện mô hình học máy

1.1 Định nghĩa Optimizer

- Thuật toán tối ưu (optimizers) trong mô hình neural network đóng vai trò quan trọng trong quá trình huấn luyện (training) để điều chỉnh các tham số như trọng số (weights) và độ lệch (biases) sao cho mô hình đạt được hiệu suất tốt nhất. Mục tiêu chính của thuật toán tối ưu là tìm giá trị nhỏ nhất hoặc lớn nhất của một hàm mất mát (loss function) thông qua việc điều chỉnh các tham số của mô hình.

- Optimizers đem lại nhiều lợi ích như:

- Hiệu suất: Sử dụng optimizer giúp mô hình đạt được hiệu suất cao hơn bằng cách điều chỉnh các tham số sao cho hàm mất mát là nhỏ nhất.
- Hội tụ nhanh chóng: Các thuật toán optimizer được thiết kế để giúp mô hình hội tụ đến giải pháp tối ưu trong thời gian ngắn.
- Giảm nguy cơ rơi vào giải pháp cục bộ: Optimizer giúp tránh nguy cơ mô hình rơi vào các giải pháp cục bộ bằng cách thực hiện các phương pháp tối ưu toàn cục.
- Linh hoạt: Sự linh hoạt của các optimizer cho phép nghiên cứu viên và lập trình viên thí nghiệm với nhiều thuật toán khác nhau để tìm ra cách tối ưu nhất cho bài toán cụ thể.

1.2 Các phương pháp Optimizers

1.2.1 Gradient Descent (GD)

Trong các bài toán tối ưu, chúng ta thường tìm giá trị nhỏ nhất của 1 hàm số nào đó, mà hàm số đạt giá trị nhỏ nhất khi đạo hàm bằng 0. Nhưng đâu phải lúc nào đạo hàm hàm số cũng được, đối với các hàm số nhiều biến thì đạo hàm rất phức tạp, thậm chí là bất khả thi. Nên thay vào đó người ta tìm điểm gần với điểm cực tiểu nhất và xem đó là nghiệm bài toán. Gradient Descent dịch ra tiếng Việt là giảm dần

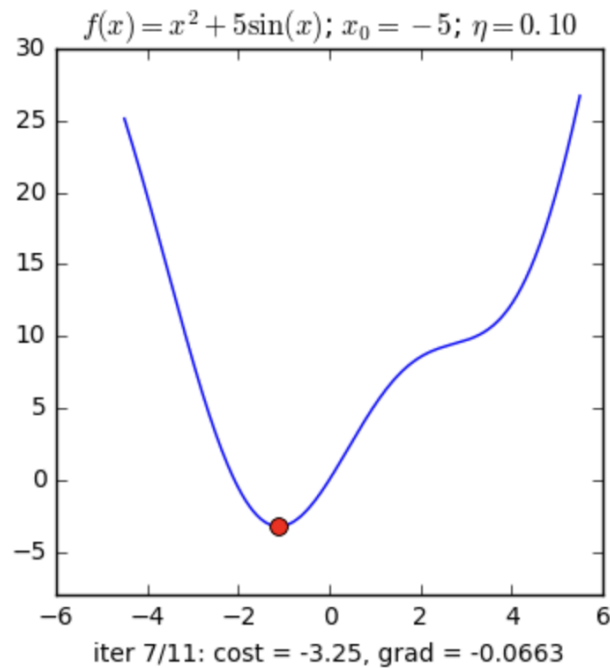
độ dốc, nên hướng tiếp cận ở đây là chọn 1 nghiệm ngẫu nhiên cứ sau mỗi vòng lặp (hay epoch) thì cho nó tiến dần đến điểm cần tìm.

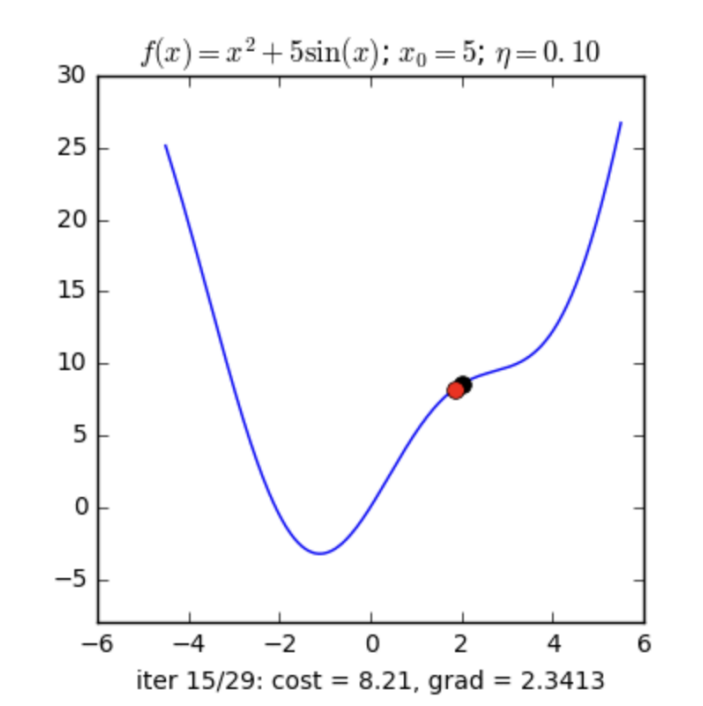
Công thức: $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} - \text{learningrate} \cdot \text{gradient}(\mathbf{x})$

Công thức trên được xây dựng để cập nhật lại nghiệm sau mỗi vòng lặp. Dấu '-' trừ ở đây ám chỉ ngược hướng đạo hàm.

Ngược hướng đạo hàm:

+ Ví dụ như đối với hàm $f(x) = 2x + 5\sin(x)$ như hình dưới thì $f'(x) = 2 + 5\cos(x)$ với $x_{\text{old}} = -4$ thì $f'(-4) < 0 \Rightarrow x_{\text{new}} > x_{\text{old}}$ nên nghiệm sẽ di chuyển về bên phải tiến gần tới điểm cực tiểu. ngược lại với $x_{\text{old}} = 4$ thì $f'(4) > 0 \Rightarrow x_{\text{new}} < x_{\text{old}}$ nên nghiệm sẽ di chuyển về bên trái tiến gần tới điểm cực tiểu.





Hình 1. Gradient Descent

Qua các hình trên ta thấy Gradient descent phụ thuộc vào nhiều yếu tố : như nếu chọn điểm x ban đầu khác nhau sẽ ảnh hưởng đến quá trình hội tụ; hoặc tốc độ học (learning rate) quá lớn hoặc quá nhỏ cũng ảnh hưởng: nếu tốc độ học quá nhỏ thì tốc độ hội tụ rất chậm ảnh hưởng đến quá trình training, còn tốc độ học quá lớn thì tiến nhanh tới đích sau vài vòng lặp tuy nhiên thuật toán không hội tụ, quanh quẩn quanh đích vì bước nhảy quá lớn.

- Ưu điểm:

- Thuật toán gradient descent cơ bản, dễ hiểu. Thuật toán đã giải quyết được vấn đề tối ưu model neural network bằng cách cập nhật trọng số sau mỗi vòng lặp.

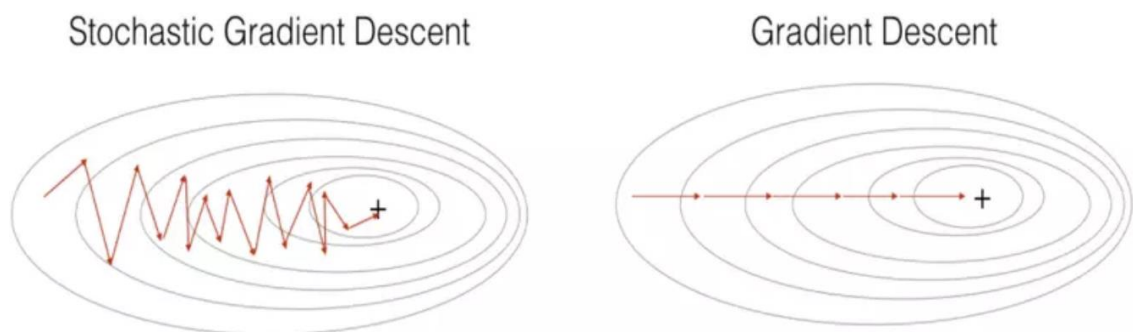
- Nhược điểm:

- Vì đơn giản nên thuật toán Gradient Descent còn nhiều hạn chế như phụ thuộc vào nghiệm khởi tạo ban đầu và learning rate.

- Ví dụ 1 hàm số có 2 global minimum thì tùy thuộc vào 2 điểm khởi tạo ban đầu sẽ cho ra 2 nghiệm cuối cùng khác nhau.
- Tốc độ học quá lớn sẽ khiến cho thuật toán không hội tụ, quanh quẩn bên đích vì bước nhảy quá lớn; hoặc tốc độ học nhỏ ảnh hưởng đến tốc độ training

1.2.2 Stochastic Gradient Descent (SGD)

Stochastic là 1 biến thể của Gradient Descent . Thay vì sau mỗi epoch chúng ta sẽ cập nhật trọng số (Weight) 1 lần thì trong mỗi epoch có N điểm dữ liệu chúng ta sẽ cập nhật trọng số N lần. Nhìn vào 1 mặt , SGD sẽ làm giảm đi tốc độ của 1 epoch. Tuy nhiên nhìn theo 1 hướng khác,SGD sẽ hội tụ rất nhanh chỉ sau vài epoch. Công thức SGD cũng tương tự như GD nhưng thực hiện trên từng điểm dữ liệu.



Hình 2. Stochastic Gradient Descent và Gradient Descent

Nhìn vào 2 hình trên, ta thấy SGD có đường đi khá là zig zắc , không mượt như GD vì 1 điểm dữ liệu không thể đại diện cho toàn bộ dữ liệu.

Chúng ta dùng SGD thay cho GD mặc dù đường đi của nó khá là zigzag vì GD có hạn chế đối với cơ sở dữ liệu lớn (vài triệu dữ liệu) thì việc tính toán đạo hàm trên toàn bộ dữ liệu qua mỗi vòng lặp trở nên cồng kềnh. Bên cạnh đó GD không phù hợp với online learning (online learning là khi dữ liệu cập nhật liên tục ,ví dụ: như thêm người dùng đăng kí thì mỗi lần thêm dữ liệu ta phải tính lại đạo hàm trên toàn bộ dữ liệu => thời gian tính toán lâu, thuật toán không online nữa).

Vì thế SGD ra đời để giải quyết vấn đề đó, vì mỗi lần thêm dữ liệu mới vào chỉ cần cập nhật trên 1 điểm dữ liệu đó thôi, phù hợp với online learning.

Một ví dụ minh họa : có 10.000 điểm dữ liệu thì chỉ sau 3 epoch ta đã có được nghiệm tốt, còn với GD ta phải dùng tới 90 epoch để đạt được kết quả đó.

- Ưu điểm:

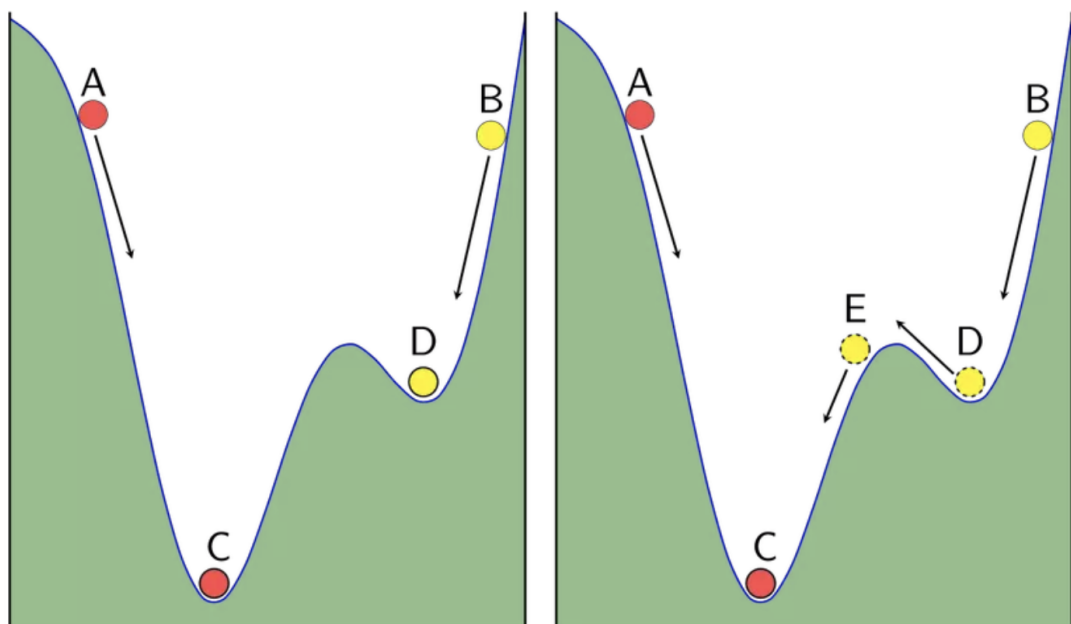
- Thuật toán giải quyết được đối với cơ sở dữ liệu lớn mà GD không làm được. Thuật toán tối ưu này hiện nay vẫn hay được sử dụng.

- Nhược điểm:

- Thuật toán vẫn chưa giải quyết được 2 nhược điểm lớn của gradient descent (learning rate, điểm dữ liệu ban đầu). Vì vậy ta phải kết hợp SGD với 1 số thuật toán khác như: Momentum, AdaGrad,..Các thuật toán này sẽ được trình bày ở phần sau.

1.2.3 Momentum

Để khắc phục các hạn chế trên của thuật toán Gradient Descent người ta dùng gradient descent with momentum.



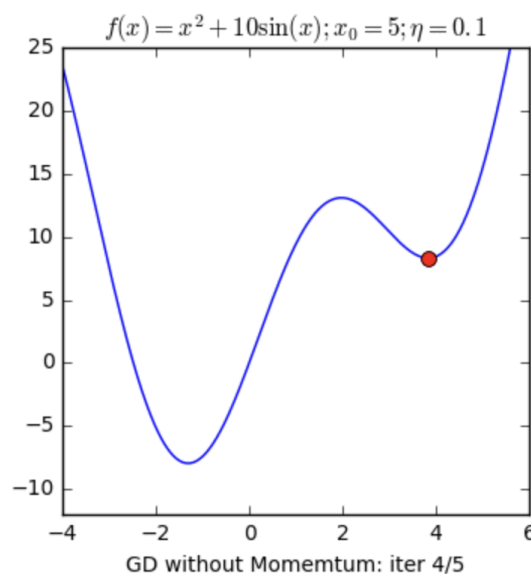
Hình 3. GD và GD with Momentum

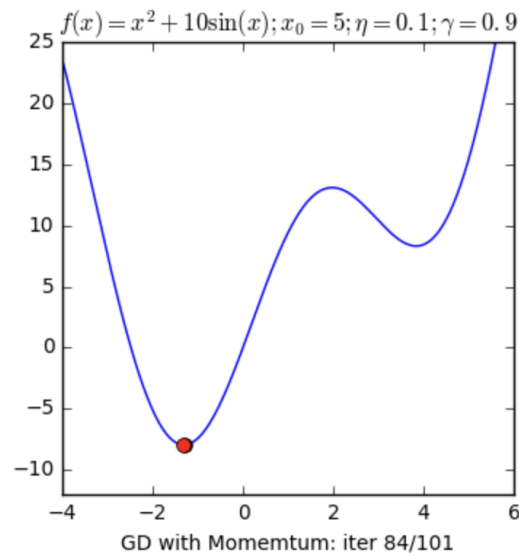
Để giải thích được Gradient with Momentum thì trước tiên ta nên nhìn dưới góc độ vật lý: Như hình b phía trên, nếu ta thả 2 viên bi tại 2 điểm khác nhau A và B thì viên bi A sẽ trượt xuống điểm C còn viên bi B sẽ trượt xuống điểm D, nhưng ta lại không mong muốn viên bi B sẽ dừng ở điểm D (local minimum) mà sẽ tiếp tục lăn tới điểm C (global minimum). Để thực hiện được điều đó ta phải cấp cho viên bi B 1 vận tốc ban đầu đủ lớn để nó có thể vượt qua điểm E tới điểm C. Dựa vào ý tưởng này người ta xây dựng nên thuật toán Momentum (tức là theo đà tiến tới).

Nhìn dưới góc độ toán học, ta có công thức Momentum:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} - (\text{gama} \cdot \mathbf{v} + \text{learningrate} \cdot \text{gradient})$$

- Trong đó :
- $\mathbf{x}_{\text{new}}$: tọa độ mới
- $\mathbf{x}_{\text{old}}$: tọa độ cũ
- gama: parameter , thường =0.9
- learningrate : tốc độ học
- gradient : đạo hàm của hàm f





Hình 4. GD with Momentum

Qua 2 ví dụ minh họa trên của hàm $f(x) = x^2 + 10\sin(x)$, ta thấy GD without momentum sẽ hội tụ sau 5 vòng lặp nhưng không phải là global minimum. Nhưng GD with momentum dù mất nhiều vòng lặp nhưng nghiệm tiến tới global minimum, qua hình ta thấy nó sẽ vượt tốc tiến tới điểm global minimum và dao động qua lại quanh điểm đó trước khi dừng lại.

- Ưu điểm:

- Thuật toán tối ưu giải quyết được vấn đề: Gradient Descent không tiến được tới điểm global minimum mà chỉ dừng lại ở local minimum.

- Nhược điểm:

- Tuy momentum giúp hòn bi vượt dốc tiến tới điểm đích, tuy nhiên khi tới gần đích, nó vẫn mất khá nhiều thời gian giao động qua lại trước khi dừng hẳn, điều này được giải thích vì viên bi có đà.

1.2.4 Adagrad

Không giống như các thuật toán trước đó thì learning rate hầu như giống nhau trong quá trình training (learning rate là hằng số), Adagrad coi learning rate là 1 tham số. Tức là Adagrad sẽ cho learning rate biến thiên sau mỗi thời điểm t .

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

Hình 5. Công thức Adagrad

Trong đó :

- η : hằng số
- g_t : gradient tại thời điểm t
- ϵ : hệ số tránh lỗi (chia cho mẫu bằng 0)
- G : là ma trận chéo mà mỗi phần tử trên đường chéo (i,i) là bình phương của đạo hàm vector tham số tại thời điểm t .

- Ưu điểm:

- Một lợi ích dễ thấy của Adagrad là tránh việc điều chỉnh learning rate bằng tay, chỉ cần để tốc độ học default là 0.01 thì thuật toán sẽ tự động điều chỉnh.

- Nhược điểm:

- Yếu điểm của Adagrad là tổng bình phương biến thiên sẽ lớn dần theo thời gian cho đến khi nó làm tốc độ học cực kì nhỏ, làm việc training trở nên đóng băng.

1.2.5 RMSprop

RMSprop giải quyết vấn đề tỷ lệ học giảm dần của Adagrad bằng cách chia tỷ lệ học cho trung bình của bình phương gradient.

$$E[g^2]_t = 0,9E[g^2]_{t-1} + 0,1g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Hình 6. Công thức RMSprop

- Ưu điểm:

- Ưu điểm rõ nhất của RMSprop là giải quyết được vấn đề tốc độ học giảm dần của Adagrad (vấn đề tốc độ học giảm dần theo thời gian sẽ khiến việc training chậm dần, có thể dẫn tới bị đóng băng).

- Nhược điểm:

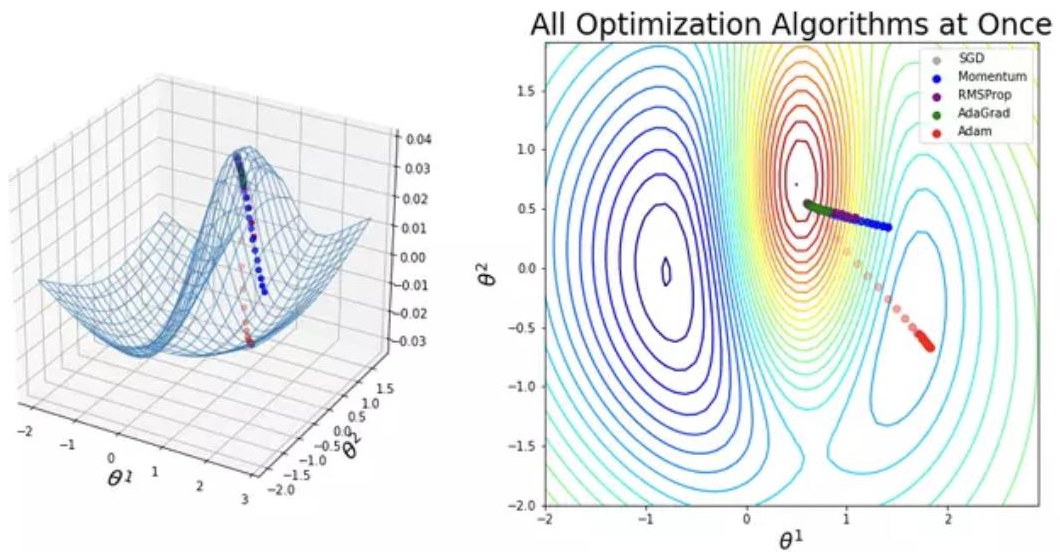
- Thuật toán RMSprop có thể cho kết quả nghiệm chỉ là local minimum chứ không đạt được global minimum như Momentum. Vì vậy người ta sẽ kết hợp cả 2 thuật toán Momentum với RMSprop cho ra 1 thuật toán tối ưu Adam. Chúng ta sẽ trình bày nó trong phần sau.

1.2.6 Adam

Như đã nói ở trên Adam là sự kết hợp của Momentum và RMSprop. Nếu giải thích theo hiện tượng vật lý thì Momentum giống như 1 quả cầu lao xuống dốc, còn Adam như 1 quả cầu rất nặng có ma sát, vì vậy nó dễ dàng vượt qua local minimum tới global minimum và khi tới global minimum nó không mất nhiều thời gian dao động qua lại quanh đích vì nó có ma sát nên dễ dừng lại hơn.

$$\begin{aligned}
\mathbf{g}_n &\leftarrow \nabla f(\boldsymbol{\theta}_{n-1}) \\
\mathbf{m}_n &\leftarrow (\beta_1/(1 - \beta_1^n)) \mathbf{m}_{n-1} + ((1 - \beta_1)/(1 - \beta_1^n)) \mathbf{g}_n \\
\mathbf{v}_n &\leftarrow (\beta_2/(1 - \beta_2^n)) \mathbf{v}_{n-1} + ((1 - \beta_2)/(1 - \beta_2^n)) \mathbf{g}_n \odot \mathbf{g}_n \\
\boldsymbol{\theta}_n &\leftarrow \boldsymbol{\theta}_{n-1} - a \mathbf{m}_n / (\sqrt{\mathbf{v}_n} + \epsilon) ,
\end{aligned}$$

Công thức Adam



Hình 7. Các thuật toán optimizers

Hiện nay optimizers hay được sử dụng nhất là 'Adam'. Qua hình trên ta thấy optimizer 'Adam' hoạt động khá tốt, tiến nhanh tới mức tối thiểu hơn các phương pháp khác. Qua bài viết này mình hy vọng các bạn có thể hiểu và làm quen với optimizer trong các bài toán Machine learning, đặc biệt là Deep learning.

2. Continual Learning và Test Production khi xây dựng một giải pháp học máy

2.1 Continual Learning

2.1.1 Định nghĩa

Continual Learning (CL) còn được gọi là Lifelong Learning là một phương pháp quan trọng trong lĩnh vực học máy, đặc biệt là khi mô hình phải đối mặt với

dữ liệu ngày càng thay đổi và phức tạp. Trong môi trường thực tế, không phải lúc nào chúng ta cũng có sẵn toàn bộ dữ liệu huấn luyện từ đầu. CL giúp mô hình duy trì và cập nhật kiến thức hiện tại khi học thêm dữ liệu mới, mà không làm mất đi kiến thức đã học từ trước.

Trong CL, mô hình phải xử lý dữ liệu một cách linh hoạt, không chỉ để học từ các dữ liệu mới mà còn để duy trì và cập nhật các kiến thức đã học trước đó.

2.1.2 Thách Thức và Giải Pháp

- Quản Lý Kiến Thức:

- Thách Thức: Mô hình có thể quên thông tin quan trọng khi học dữ liệu mới.
- Giải Pháp: Sử dụng kỹ thuật Regularization như Elastic Weight Consolidation để giảm thiểu việc quên thông tin quan trọng và bảo toàn trọng số quan trọng.

- Catastrophic Forgetting:

- Thách Thức: Học dữ liệu mới có thể dẫn đến việc quên hoàn toàn dữ liệu cũ.
- Giải Pháp: Sử dụng kỹ thuật Memory Replay, trong đó mô hình giữ lại và sử dụng lại các mẫu dữ liệu cũ để giữ lại kiến thức trước đó.

2.1.3 Các phương pháp Continual Learning

- Regularization Techniques:

- Mục Tiêu: Giảm thiểu độ lớn của trọng số và duy trì độ nhạy cảm của chúng đối với các features quan trọng.
- Kỹ Thuật: Elastic Weight Consolidation, đặt trọng số lớp để quản lý ảnh hưởng của các lớp khác nhau.

- Memory Replay:

- Mục Tiêu: Giữ lại và sử dụng lại mẫu dữ liệu cũ để giảm thiểu Catastrophic Forgetting.
- Kỹ Thuật: Lưu trữ và trích xuất mẫu dữ liệu cũ khi mô hình học dữ liệu mới.

- Dynamic Architectures:

- Mục Tiêu: Tối ưu hóa cấu trúc mô hình để hỗ trợ việc học mới.
- Kỹ Thuật: Tăng cường số lượng lớp, đơn vị hoặc điều chỉnh kiến trúc mô hình dựa trên dữ liệu mới.

2.2 Test Production

2.2.1 Định nghĩa

Test Production là quá trình tạo ra và thực hiện bài kiểm thử để đảm bảo mô hình hoạt động hiệu quả trong môi trường thực tế. Điều này đặc biệt quan trọng khi triển khai mô hình vào sản xuất và cần đảm bảo rằng nó đáp ứng đúng yêu cầu và chuẩn mực.

2.2.2 Bước Thiết Kế Test Production

- Tạo Tập Dữ Liệu Kiểm Thử:

- Quy Trình: Chọn hoặc tạo ra một tập dữ liệu kiểm thử đại diện cho các điều kiện thực tế mà mô hình sẽ đối mặt.
- Mục Tiêu: Đảm bảo rằng mọi khía cạnh của mô hình được kiểm thử, từ tính đồng đều đến độ chính xác.

- Kiểm Thử Tự Động:

- Quy Trình: Sử dụng công cụ kiểm thử tự động để thực hiện các bài kiểm thử định kỳ.
- Mục Tiêu: Tự động hóa quy trình kiểm thử để đảm bảo tính ổn định và chính xác.

- Kiểm Thử Hiệu Suất:

- Quy Trình: Đánh giá hiệu suất của mô hình dựa trên các tiêu chí đã đặt ra.
- Mục Tiêu: Đảm bảo rằng mô hình đáp ứng các yêu cầu về thời gian phản hồi, tải trọng, và hiệu quả tổng thể.

2.2.3 Lợi ích của Test Production

- Đảm Bảo Tính Ổn Định:

- Ý Nghĩa: Mô hình cần duy trì độ ổn định trong môi trường sản xuất.
- Lợi Ích: Đảm bảo rằng mô hình hoạt động hiệu quả dưới mọi điều kiện.

- Phát Hiện Lỗi Sớm:

- Ý Nghĩa: Phát hiện và sửa lỗi từ giai đoạn phát triển.
- Lợi Ích: Giảm rủi ro và chi phí sửa lỗi ở giai đoạn muộn.

- Dữ Liệu Dự Báo Chính Xác:

- Ý Nghĩa: Cung cấp dữ liệu chính xác và đáng tin cậy cho các ứng dụng thực tế.
- Lợi Ích: Đảm bảo rằng mô hình đưa ra dự báo chính xác và hữu ích.

2.3 Tổng kết

Kết hợp Continual Learning và Test Production là chiến lược quan trọng để xây dựng và duy trì mô hình học máy trong môi trường thực tế. Continual Learning giúp mô hình không chỉ học từ dữ liệu mới mà còn duy trì và cập nhật kiến thức đã học trước đó. Test Production đảm bảo rằng mô hình hoạt động hiệu quả và chính xác khi triển khai, với mục tiêu đảm bảo tính ổn định và đáng tin cậy trong môi trường sản xuất.

TÀI LIỆU THAM KHẢO

1. *Introduction to Continual Learning*. (2021). Retrieved from continualAI Wiki.
2. Trức, T. T. (2020, 10 17). *Optimizer- Hiểu sâu về các thuật toán tối ưu (GD,SGD,Adam,..)*. Retrieved from viblo.